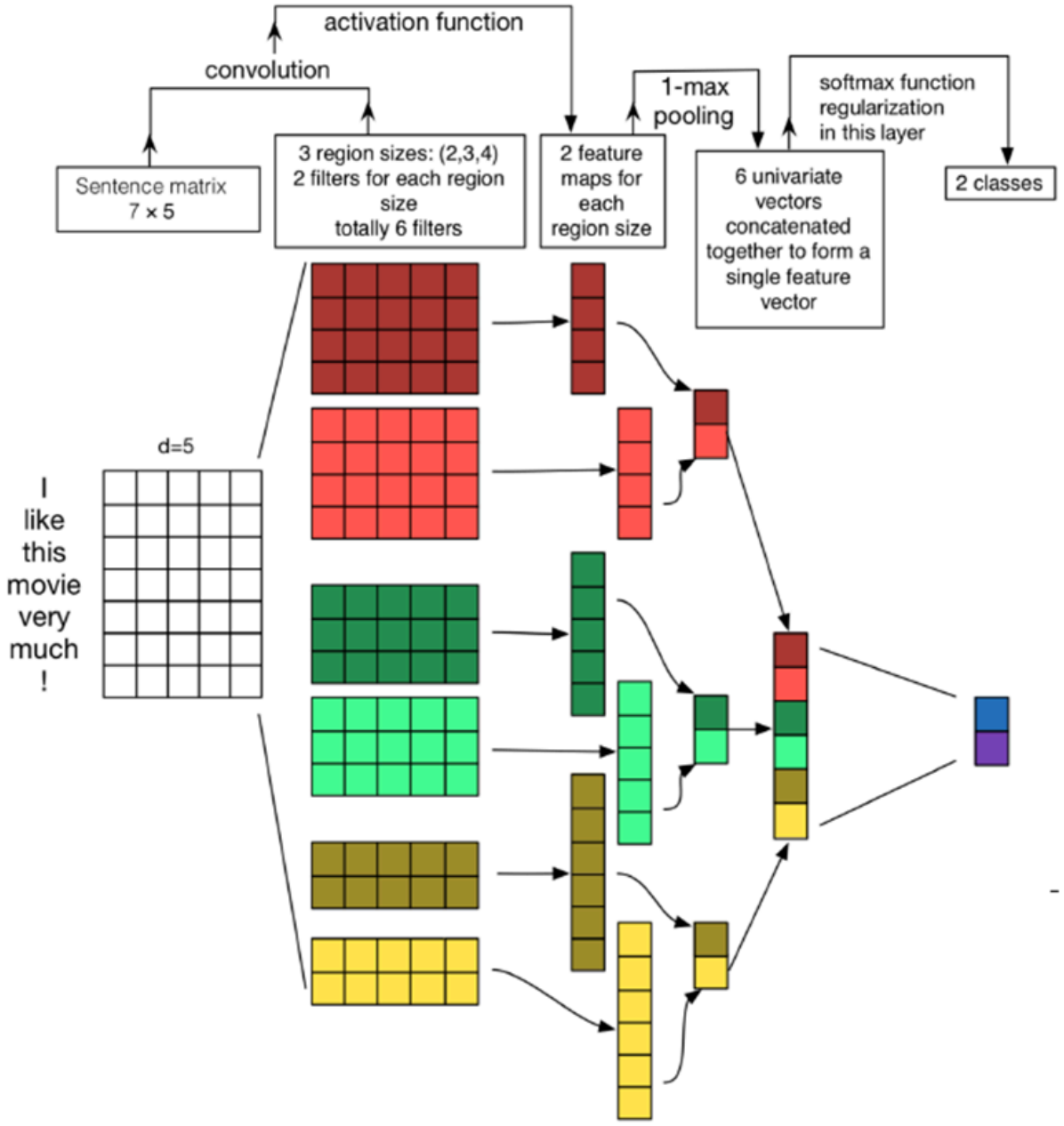
TextCNN情感分类

一、实验目标

- 用 TextCNN 模型自动判断电影评论情感是**正面（pos）*还是*负面（neg）**
- 掌握文本分类全流程：数据→预处理→模型搭建→训练→评估→推理
- 理解 CNN 如何用于文本任务，学会用 MindSpore 搭深度学习模型

Model	MR	SST-1	SST-2	Subj	TREC	CR	MPQA
CNN-rand	76.1	45.0	82.7	89.6	91.2	79.8	83.4
CNN-static	81.0	45.5	86.8	93.0	92.8	84.7	**89.6**
CNN-non-static	**81.5**	48.0	87.2	93.4	93.6	84.3	89.5
CNN-multichannel	81.1	47.4	**88.1**	93.2	92.2	**85.0**	89.4
RAE (Socher et al., 2011)	77.7	43.2	82.4	–	–	–	86.4
MV-RNN (Socher et al., 2012)	79.0	44.4	82.9	–	–	–	–
RNTN (Socher et al., 2013)	–	45.7	85.4	–	–	–	–
DCNN (Kalchbrenner et al., 2014)	–	48.5	86.8	–	93.0	–	–
Paragraph-Vec (Le and Mikolov, 2014)	–	**48.7**	87.8	–	–	–	–
CCAE (Hermann and Blunsom, 2013)	77.8	–	–	–	–	–	87.2
Sent-Parser (Dong et al., 2014)	79.5	–	–	–	–	–	86.3
NBSVM (Wang and Manning, 2012)	79.4	–	–	93.2	–	81.8	86.3
MNB (Wang and Manning, 2012)	79.0	–	–	**93.6**	–	80.0	86.3
G-Dropout (Wang and Manning, 2013)	79.0	–	–	93.4	–	82.1	86.1
F-Dropout (Wang and Manning, 2013)	79.1	–	–	**93.6**	–	81.9	86.3
Tree-CRF (Nakagawa et al., 2010)	77.3	–	–	–	–	81.4	86.1
CRF-PR (Yang and Cardie, 2014)	–	–	–	–	–	82.7	–
SVM_S (Silva et al., 2011)	–	–	–	–	**95.0**	–	–



二、实验输入 / 输出

- 输入：英文电影评论数据集（`rt-polarity.neg` 负面、`rt-polarity.pos` 正面）
- 输出：训练好的 TextCNN 模型 + 情感分类准确率 + 自定义句子预测结果

三、实验内容TextCNN 核心原理（通俗版 + 专业版）

结合你的 PPT 和实验代码，TextCNN 是**专门用于文本分类的轻量卷积神经网络**，核心是用卷积提取文本的关键特征。

1. 核心流程（6 步）

1. **文本预处理**

   把句子分词、去掉标点 / 数字、构建单词词典，将文字转为数字索引。

2. **词嵌入层**

   将数字索引转为固定维度的词向量（实验用 40 维），把文本变成模型能处理的矩阵。

3. **一维卷积层（核心）**

用3 种卷积核（窗口 3/4/5），分别提取句子中3 词、4 词、5 词组合的 n-gram 特征（比如 "very good"、"so boring"），对应 PPT 里的1D 卷积。

4. **全局最大池化**

   每个卷积核输出取最大值，保留最关键的文本特征，忽略位置差异。

5. **Dropout 防过拟合**

   随机丢弃 50% 神经元，避免模型只记住训练数据。

6. **全连接 + 分类**

   拼接 3 种卷积特征，通过全连接层 + Softmax 输出正面 / 负面二分类结果。

2. 模型结构（实验代码对应）

```
词嵌入 → 卷积(3/4/5) → 最大池化 → 特征拼接 → Dropout → 全连接 → 分类
```

四、实验过程

我把手册的 12 步简化为**4 大阶段**，直接跟着做就能跑通。

阶段 1：创建 ModelArts Notebook 环境

1. 进入华为云 ModelArts，新建 Notebook
2. 配置选择（严格匹配实验要求）
   - 名称：`textcnn`
   - 工作环境：`tensorflow1.15-mindspore1.5.1-cann5.0.3-euler2.8-aarch64`
   - 规格：`Ascend:1*Ascend910|CPU:24核96GB`
   - JupyterLab 版本：**选 4**（3 即将下线）
3. 启动后，内核选择：`MindSpore-python3.7-aarch64`

阶段 2：上传实验数据

1. 下载实验 NLP 数据包，获取`data`文件夹
2. 在 Notebook 新建

   ```
   data
   ```

   文件夹，上传两个文件：
   - `rt-polarity.neg`（负面评论）
   - `rt-polarity.pos`（正面评论）

阶段 3：逐段运行代码（复制手册代码即可）

1. **导入依赖库**

   加载 mindspore、numpy、os 等 NLP 所需工具包

2. **设置超参数**

   批次 64、训练 4 轮、句子最大长度 51、词向量 40 维

3. **配置运行环境**

指定 Ascend 芯片 + 图模式，加速训练

```
context.set_context(mode=context.GRAPH_MODE, device_target=cfg.device_target, device_id=cfg.device_id)
```

4. **数据预览**

打印前 5 条正负评论，确认数据正常

```
# 数据预览
with open("./data/rt-polarity.neg", 'r', encoding='utf-8') as f:
        print("Negative reivews:")
        for i in range(5):
            print("[{0}]:{1}".format(i,f.readline()))
with open("./data/rt-polarity.pos", 'r', encoding='utf-8') as f:
        print("Positive reivews:")
        for i in range(5):
            print("[{0}]:{1}".format(i,f.readline()))
```

[5]

```
Negative reivews:
[0]:simplistic , silly and tedious .

[1]:it's so laddish and juvenile , only teenage boys could possibly find it funny .

[2]:exploitative and largely devoid of the depth or sophistication that would make wat

[3]:[garbus] discards the potential for pathological study , exhuming instead , the sk

[4]:a visually flashy but narratively opaque and emotionally vapid exercise in style a

Positive reivews:
[0]:the rock is destined to be the 21st century's new " conan " and that he's going to

[1]:the gorgeously elaborate continuation of " the lord of the rings " trilogy is so h

[2]:effective but too-tepid biopic

[3]:if you sometimes like to go to the movies to have fun , wasabi is a good place to

[4]:emerges as something rare , an issue movie that's so honest and keenly observed th
```

```
Negative reivews:
[0]:simplistic , silly and tedious .

[1]:it's so laddish and juvenile , only teenage boys could possibly find it funny .

[2]:exploitative and largely devoid of the depth or sophistication that would make watching such a graphic treatment of the crimes bearable .

[3]:[garbus] discards the potential for pathological study , exhuming instead , the skewed melodrama of the circumstantial situation .
```

```
[4]:a visually flashy but narratively opaque and emotionally vapid exercise
in style and mystification .

Positive reivews:
[0]:the rock is destined to be the 21st century's new " conan " and that he's
going to make a splash even greater than arnold schwarzenegger , jean-claud
van damme or steven segal .

[1]:the gorgeously elaborate continuation of " the lord of the rings "
trilogy is so huge that a column of words cannot adequately describe co-
writer/director peter jackson's expanded vision of j . r . r . tolkien's
middle-earth .

[2]:effective but too-tepid biopic

[3]:if you sometimes like to go to the movies to have fun , wasabi is a good
place to start .

[4]:emerges as something rare , an issue movie that's so honest and keenly
observed that it doesn't feel like one .
```

5. **定义数据预处理类**

   完成分词、去噪、构建词典、转数字索引、划分训练 / 测试集

```
class Generator():
    def __init__(self, input_list):
    def __getitem__(self,item):
    def __len__(self):
```

```
class MovieReview:
    def __init__(self, root_dir, maxlen, split):
    def read_data(self, filePath):
    def text2vec(self, maxlen):
    def split_dataset(self, split):
    def get_dict_len(self):
    def create_train_dataset(self, epoch_size, batch_size):
    def create_test_dataset(self, batch_size):
```

6. **生成数据集**

   批量加载训练 / 测试数据，适配模型输入

```
instance = MovieReview(root_dir=cfg.data_path, maxlen=cfg.word_len,
split=0.9)
dataset =
instance.create_train_dataset(batch_size=cfg.batch_size,epoch_size=cfg.epoch_
size)
batch_num = dataset.get_dataset_size()
```

```
vocab_size:18848
{'data': Tensor(shape=[64, 51], dtype=Int32, value=
[[  59,  764, 1262 ...    0,    0,    0],
 [ 107, 6304,   32 ...    0,    0,    0],
 [ 297,  111,  366 ...    0,    0,    0],
 ...
 [ 646, 4868,   10 ...    0,    0,    0],
 [4021,  607,    4 ...    0,    0,    0],
 [ 128, 2118,  275 ...    0,    0,    0]]), 'label': Tensor(shape=[64], dtype=Int32, value= [1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1,
 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1,
 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1]])}
[  107  6304    32  2806  6175 16613  5309    10  1240  5312   735 13729
     0  4342   126     0  6346    32     0   273     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     0     0     0     0
     0     0     0]
```

7. **设置学习率**

用热身 + 衰减的学习率，让训练更稳定

```python
learning_rate = []
warm_up = [1e-3 / math.floor(cfg.epoch_size / 5) * (i + 1) for _ in
range(batch_num)
           for i in range(math.floor(cfg.epoch_size / 5))]
shrink = [1e-3 / (16 * (i + 1)) for _ in range(batch_num)
          for i in range(math.floor(cfg.epoch_size * 3 / 5))]
normal_run = [1e-3 for _ in range(batch_num) for i in
              range(cfg.epoch_size - math.floor(cfg.epoch_size / 5)
                    - math.floor(cfg.epoch_size * 2 / 5))]
learning_rate = learning_rate + warm_up + normal_run + shrink
```

8. **定义 TextCNN 模型**

搭建嵌入→卷积→池化→全连接的完整结构

```python
net = TextCNN(vocab_len=instance.get_dict_len(), word_len=cfg.word_len,
              num_classes=cfg.num_classes, vec_length=cfg.vec_length)
```

```python
# ========== 补充文档中没有的打印模型完整结构代码 ==========
print("===== TextCNN 模型完整结构 =====")
print(net)
```

```
# ========== 打印模型完整结构（核心） ==========
print("===== TextCNN 模型完整结构 =====")
print(net)
Python

===== TextCNN 模型完整结构 =====
TextCNN<
  (embedding): Embedding<vocab_size=18848, embedding_size=40, use_one_hot=False, embedding_table=Parameter (name=embedding.embedding_table, shape=(18848, 40), dtype=F
  (layer1): SequentialCell<
    (0): Conv2d<input_channels=1, output_channels=96, kernel_size=(3, 40), stride=(1, 1), pad_mode=pad, padding=1, dilation=(1, 1), group=1, has_bias=True, weight_ini
        -8.42232071e-03  1.15296040e-02]
       [ 9.01064929e-03  1.65704805e-02 -1.86909419e-02 ...  1.66803468e-02
        -4.16656258e-03  6.12955866e-03]
       [-1.27170570e-02  1.01673836e-02  1.43217470e-03 ...  1.56860231e-04
         1.03003401e-02  2.46949517e-03]]]


     [[[-1.66447368e-02  1.08563667e-02 -1.10515840e-02 ...  7.15558045e-03
         1.19528854e-02 -4.26250137e-03]
       [-8.61144532e-03  2.56031100e-03 -1.23684097e-03 ... -1.39893247e-02
        -1.56398979e-03  1.59128034e-03]
       [ 5.41212177e-03  2.45867553e-03 -1.54814059e-02 ... -1.12963710e-02
        -4.40699933e-03  1.19477825e-03]]]


     [[[-3.62969330e-03  6.38748286e-03 -1.56826980e-03 ... -3.08101228e-03
        -6.61245873e-03  1.23112053e-02]
       [-1.13485884e-02 -2.20255256e-02 -4.64073569e-03 ... -4.21455316e-03
        -1.13025187e-02 -7.50623550e-03]
       [-2.20974488e-03  2.84096641e-05  2.36358810e-02 ...  1.52905867e-03
 ...
    >
  (fc): Dense<input_channels=288, output_channels=2, has_bias=True>
  (drop): Dropout<keep_prob=0.5>
  >
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

9. **配置训练参数**

   优化器（Adam）、损失函数（交叉熵）、模型保存

10. **启动训练**

    运行 4 轮，打印损失值，自动保存模型文件

```
model.train(cfg.epoch_size, dataset, callbacks=[time_cb, ckpoint_cb, loss_cb])
print("train success")
```

```
epoch: 1 step: 596, loss is 1.0157965198273245e+29
epoch time: 4818.812 ms, per step time: 8.085 ms
epoch: 2 step: 596, loss is 1.0784719566826297e+29
epoch time: 4805.645 ms, per step time: 8.063 ms
epoch: 3 step: 596, loss is 8.472112475980865e+28
epoch time: 4809.743 ms, per step time: 8.070 ms
epoch: 4 step: 596, loss is 7.158148475397711e+28
epoch time: 4813.004 ms, per step time: 8.076 ms
train success
```

11. **模型评估**

    加载训练好的模型，在测试集计算分类准确率

12. **在线推理**

    输入自定义句子（如下，输出正负情感）

```
review_en = "the movie is so boring"
inference(review_en)
```

```
[27]

...   Negative comments
```

阶段 4：查看实验结果

1. 训练：损失值逐步下降，说明模型在学习
2. 评估：输出测试集准确率（正常能到 75% 以上）
3. 推理：自定义句子成功输出**Positive/Negative comments**

五、实验分析

1.环境选**mindspore1.7.0-cann5.1.0-py3.7-euler2.8.3**版本(华北北京四)，和实验代码完全兼容

2.JupyterLab version3 版本 虽然version42026 年 4 月下线会报错

3.数据路径必须是`./data/`，否则代码读不到文件

4.步骤 8 定义TextCNN处

缩进错误

```
def _weight_variable(shape, factor=0.01):
    init_value = np.random.randn(*shape).astype(np.float32) * factor
    return Tensor(init_value)
```

步骤 8　定义 TextCNN

模型类定义了模型结构搭建、训练、评估、加载离线模型、在线推理函数。

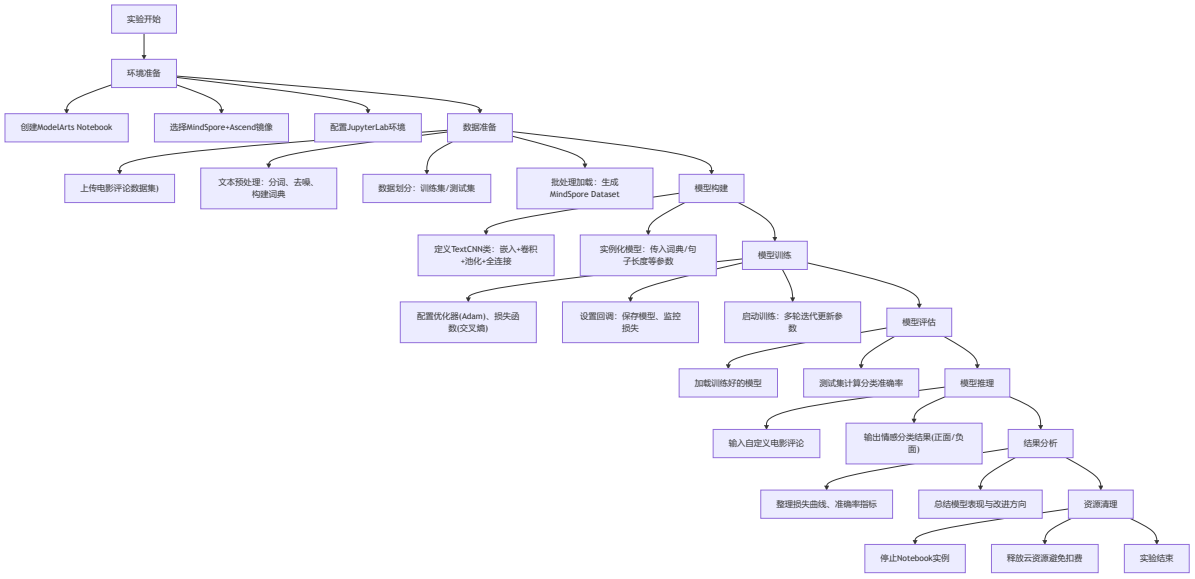
输入：

```
def _weight_variable(shape, factor=0.01):
    init_value = np.random.randn(*shape).astype(np.float32) * factor
    return Tensor(init_value)
```

5.定义 TextCNN 模型后补充代码，查看神经网络概括

```
print("===== TextCNN 模型完整结构 =====")
print(net)
```

6.实验说明流程图



7.实验收获

- 掌握了 TextCNN 文本分类的基本原理与网络结构
- 学会使用 MindSpore 框架实现模型定义、训练与评估
- 能够熟练在 ModelArts 云平台上运行深度学习实验
- 提升了代码调试与错误排查能力
- 养成了规范使用云资源、及时保存与导出文件的习惯